

MRK18 · THE BITTER TRUTH · KNOWS · TELLS · FIXES

MRK18 — Virtual Device: Feasibility & Build Research

Can we make a device that watches a real campaign and speaks the bitter truth out loud? Yes. Here's exactly how — and what to build first.

Researched June 2026.

THE HEADLINE ANSWER

Yes, it's fully buildable — and the off-the-shelf parts to do it exist today and are cheap. The image you uploaded (the Elecrow ESP32 AI camera board, \$24.90) is literally one of the building blocks. A board like the Hiwonder WonderLLM (\$29.99) already combines a camera + mic array + speaker + always-on wake-word chip and connects to cloud LLMs out of the box.

But — and this is the part that decides whether you win or die — **the "device" is the demo magnet, not the product.** The product is software. Three different AI hardware startups proved this the hard way in 2025–26. So the real question isn't "can we build the hardware" (you can, cheaply). It's "what do we actually ship first." Answer at the bottom: **a web app (PWA) first. Not native app. Not hardware-to-customers.**

PART 1 — IS THE HARDWARE POSSIBLE? (yes, and it's a solved problem)

What your device needs to do — see a screen/campaign, hear/speak, think — maps onto a well-trodden 2026 hardware stack:

Function	What does it	Reality in 2026
See the campaign	Camera (2MP is plenty for a screen)	On the Elecrow/ESP32-CAM boards you already have
Hear you	Mic array	On-board on WonderLLM-class boards
Speak the bitter truth	Speaker + TTS	On-board speaker; voice generated in the cloud
Wake on command	Dedicated voice chip (e.g. CI1302)	Always-on wake-word, offline
Think	The LLM	Does NOT run on the device — runs in your cloud

The key architectural fact: the ESP32 is *not* the brain. It's the senses. The standard pattern (used by the open-source **XiaoZhi** framework that these boards run on) is:

Device = captures audio/image, detects wake-word, streams to cloud → **Your server** = does the speech-to-text, the LLM reasoning, the text-to-speech → streams the spoken answer back to the device's speaker.

This is why a ₹2,500 microcontroller can feel like a smart CMO: it's a thin client in front of your real software. **That means 95% of your engineering is the cloud software — which you're building anyway for the watchdog.** The device is a shell around it.

Realistic hardware paths for a 2-person, pre-funding team

1. **Now (demo): no custom hardware at all.** Use an off-the-shelf XiaoZhi/WonderLLM/Elecrow board running open firmware, pointed at your cloud. This is almost certainly what your working demo already is. Perfect for filming.
2. **Design-partner phase: still off-the-shelf.** Hand a handful of these boards (flashed with your firmware) to a few design partners as a "founding device." No factory, no supply chain, no certification.
3. **Post-funding only: custom hardware.** Injection-molded casing, PCB design, BIS certification (India), manufacturing, support, returns. **Do not touch this before funding.** This is the exact step that killed the startups below.

PART 2 — THE CAUTIONARY TALE (read this twice)

Three 2025–26 AI hardware flops, and what each teaches MRK18:

- **Humane AI Pin** — raised \$230M, sold to HP for \$116M after shipping <10,000 units. **Lesson: hardware complexity (factories, supply chains, support, returns) is brutally expensive and slow.**

- **Rabbit R1** — sold 100,000 units on viral hype, then mass returns, reportedly struggling to make payroll. **Lesson: overpromising vs. what shipped. The demo wrote a check the product couldn't cash.**
- **The shared failure** — *all three confused viral demos and waitlists with product-market fit.* Pre-order excitement is not validation.

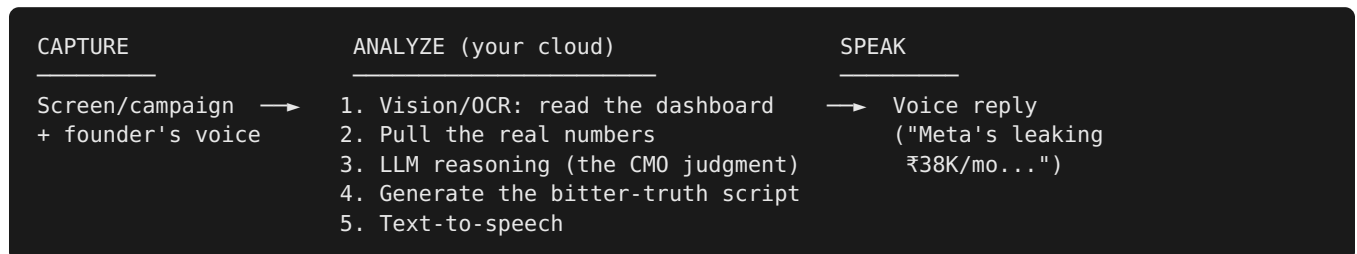
The single most important line in this whole document, for you:

Integration beats standalone devices. AI doesn't need a new gadget — it needs to improve the tools founders already use.

This is *good news* for MRK18. It means your real product (software watchdog the founder uses today) is the right thing to sell, and the device is the thing that gets attention. You already arrived at this in the distribution kit ("show the demo, sell the access"). The hardware research confirms it: **lead with the device, deliver the software, build custom hardware only after funding.**

PART 3 — HOW THE SOFTWARE ACTUALLY WORKS (the real product)

This is the pipeline behind both the device *and* the app. Same brain, different front door.



The two ways to build the voice part (2026 facts)

Option A — Speech-to-speech (one model, e.g. OpenAI Realtime API). - Fastest: sub-300ms first-token latency in US/EU; feels human. - Most expensive: ~₹25-30/min all-in (~\$0.30/min). Cost scales fast with call volume. - Best for: the *live* "your CMO is calling" experience.

Option B — Chained pipeline (separate STT → LLM → TTS). - Cheaper: e.g. Deepgram Flux STT at ~\$0.0065/min; pick your own cheap LLM; pick a TTS. - Slower: typically 600-900ms median latency (still fine for a CMO call). - Best for: controlling cost, and **swapping in Indian/sovereign models** (your moat — Sarvam, etc.). The chained design is what lets you stay model-independent.

Recommendation for MRK18: chained pipeline (Option B) as your default — it's cheaper at the volume you're targeting *and* it's the only design that lets you honor the "sovereign Indian

models" positioning from your deck. Use speech-to-speech only for the premium live-call moment if latency ever feels off.

Note: these are the strongest current voice stacks but pricing/latency shift monthly — re-check before you commit a provider. Cost per founder-call is your key unit-economics number; model it before you scale.

PART 4 — APP vs WEBSITE: WHAT TO BUILD FIRST

Build a web app first — specifically a PWA (Progressive Web App). Do NOT build a native iOS/Android app yet. This is the clearest call in the whole document.

Why PWA wins for MRK18 specifically

	PWA (web app)	Native app
Time to MVP	~8-16 weeks	16-40 weeks (two platforms)
Cost	~30-40% of native, same result if no heavy hardware features	\$80K-250K+, two parallel teams
Discovery	Found via Google search — opens instantly, no install	Play Store friction: search → download → wait
India fit	Beats storage limits, data-cost worries, install friction	Loses users at every install step
Your buyer	A founder who'll click a link from your DM/post and use it in 10 seconds	A founder who won't install an unknown app

For a **search-and-content-driven B2B SaaS in India** — which is exactly your build-in-public + Bitter Truth Weekly motion — a PWA lets a founder find you on Google or click your DM link and *use the product immediately*, no install. That's a perfect match for how you're already getting distribution.

A PWA also installs to the home screen, sends push notifications ("Your weekly bitter truth is ready"), and works offline-ish — so it *feels* like an app without the native cost or the Play Store wall.

So what's the build order?

1. **Web app (PWA)** — the watchdog dashboard + the "your CMO is calling" voice flow. This is the product you sell on 22 Jun. *Build first.*
2. **The device** — stays as an off-the-shelf demo / founding-partner perk, pointing at the *same* cloud. No separate product to build.
3. **Native app** — only if PWA usage data later proves founders want it. Probably never urgent.

4. **Custom hardware** — post-funding only.

WHAT TO DO FIRST — IN ORDER

1. **Confirm your demo board + firmware** (likely XiaoZhi-based on an ESP32-S3 cam board). Get the 30-sec clip filmed — that's your distribution hero.
 2. **Build the PWA**, not a native app. Watchdog dashboard + voice call. Ship the watchdog on 22 Jun.
 3. **Wire the voice pipeline as a chained STT→LLM→TTS stack** so you can run Indian/sovereign models and control cost. Model your cost-per-call before scaling.
 4. **Keep the device off-the-shelf** through design-partner phase. No factory until you're funded.
 5. **Never let a paying founder believe custom hardware ships day one.** Device = demo + queue; software = what they get.
-

SOURCES

- Hiwonder WonderLLM ESP32-S3 smart chat module — CNX Software
- XiaoZhi AI Dev — ESP32 voice robot framework
- Building a DIY ESP32-S3 AI Voice Assistant with XiaoZhi — DEV Community
- esp32-cam-ai: MCP server for ESP32-CAM + LLM — GitHub
- AI Product Failures 2026: Sora, Humane & Rabbit R1 — Digital Applied
- Anatomy of a Failure: The Humane AI Pin — Bossa Research / Medium
- Real-Time vs Turn-Based Voice Agents in 2026 — Softcery
- Introducing gpt-realtime and Realtime API updates — OpenAI
- Best Speech APIs in 2026: TTS, STT Compared — Gadium
- PWA vs Native App: Pros and Cons in 2026 — instinctools
- Progressive Web Apps vs Native Apps for Indian Businesses — Aurtos Studio